

# 8주차 · 한 점을 중심으로 도는 로이터링

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

## ★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 본 과목 담당 교수가 직접 강의한 것이며, 본 과목이 전제하는 배경 지식에 해당한다. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 된다. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

| # | 과목                                            | 수준  | 언어  | 영상   | 자료·코드 |
|---|-----------------------------------------------|-----|-----|------|-------|
| 1 | 제어공학 — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대     | 학부  | 한국어 | 재생목록 | 드라이브  |
| 2 | 제어시스템설계 — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현    | 학부  | 한국어 | 재생목록 | 드라이브  |
| 3 | 캡스톤디자인 — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지   | 학부  | 한국어 | 재생목록 | 드라이브  |
| 4 | 제어공학특론 — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림 | 대학원 | 영어  | 재생목록 | 드라이브  |
| 5 | 센서신호처리 및 융합 — 센서 모델, 잡음, 추정, 다중센서 융합          | 대학원 | 영어  | 재생목록 | 드라이브  |

MATLAB-Simulink 가 처음이라면 6주차 실습 전에 아래를 끝낼 것. 본 과목의 제어가 실습은 전부 Simulink 로 진행한다. Onramp 는 무료이며 각각 몇 시간이면 끝난다.

| 도구       | 시작 지점                                                       |
|----------|-------------------------------------------------------------|
| MATLAB   | <a href="#">MATLAB Onramp · Core MATLAB Skills</a>          |
| Simulink | <a href="#">Simulink Onramp</a> · 담당 교수 Simulink 강의 1부 · 2부 |

- 과목: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- 이번 주차 학습 내용: 한 점을 중심으로 원을 그리며 돌게 만든다. 방향·속도·반경을 바꾼다

## ★ 시작 전 확인

- 7주차 `W07_0_offline.slx` 가 동작했어야 함
- 배포 폴더 `W08_simulink/` , `W08_matlab/`
- 논문 PDF: [50-원본자료/](#) 의 `Unmanned Aircraft Vector Field Path Following with Arrival Angle Control.pdf`

## ! 7주차에서 바뀌는 것은 유도 한 단뿐이다

내부루프(헤딩 P-D + 속도 PI), 추진기 모델, 운동모델, 게인이 전부 그대로다. 유도만 LOS 에서 벡터필드로 바꾼다. 이것이 GNC 를 층으로 나눠 설계하는 이유다.

## 이 주차를 마치면 할 수 있어야 하는 것

1. 로이터링이 무엇이고 왜 필요한지 설명하기
2. 웨이포인트로 원을 흉내내면 왜 안 되는지 설명하기
3. 벡터필드라는 발상을 설명하기
4. 논문의 식 (9)·(13) 을 읽고 코드와 대조하기
5. `p_c` 의 부호가 회전 방향, 크기가 수렴 성향임을 실험으로 보이기
6. 식 (28) 로 `p_c` 를 계산해서 정하기
7. 남은 반경 오차의 원인을 세 가지로 분해하기
8. 회전수를 세어 임무를 끝내기

## 준비물

| 항목    | 내용                                                                                                                                                         |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 환경    | 1~7주차 그대로                                                                                                                                                  |
| 배포 파일 | <a href="#">W08_simulink/</a> (모델 2개 + 스크립트 7개: setup · plot · animate · vectorfield · vrx_run · build · tidy_layout), <a href="#">W08_matlab/</a> (원본 참고) |
| 논문    | <a href="#">50-원본자료/</a> PDF                                                                                                                               |
| 중간 발표 | 팀별 Term Project 제안서 + 전반부 데모                                                                                                                               |

### ★ 이번 주는 중간 발표 주간이다

- 평가의 15% — [평가와-팀운영](#) 참조
- 제출물: Term Project 제안서 ([Term-Project-명세](#) 의 양식)
- 데모: 1~7주차에 만든 것 중 팀이 고른 하나를 실제로 돌려 보인다
- 발표는 수업 앞 60분에 하고, 로이터링 실습은 남은 시간에 진행한다.  
이론(1부)은 미리 읽어 오는 것을 전제로 요약만 다룬다

# 1부 · 이론

## 1-1. 로이터링이란

- 한 점을 중심으로 원을 그리며 계속 도는 것
- 영어로 loitering — "어슬렁거리다"

### 왜 필요한가

| 상황           | 왜 도는가                   |
|--------------|-------------------------|
| 표적 감시        | 한 지점을 계속 보면서 거리를 유지해야 함 |
| 대기 (holding) | 다음 지시가 올 때까지 자리를 지켜야 함  |
| 통신 중계        | 일정 반경 안에 머물러야 함         |
| 수색           | 중심에서 나선형으로 넓혀 가며 훑음     |

- 배는 헬리콥터가 아니라 **제자리에 틀 수 없다**
- 멈추면 조류·바람에 밀린다 → **돌면서 자리를 지키는 것**이 현실적인 해법
- 제자리 유지(DP)는 13주차에서 따로 다룬다

## 1-2. 웨이포인트로 원을 흉내내면 안 되는가

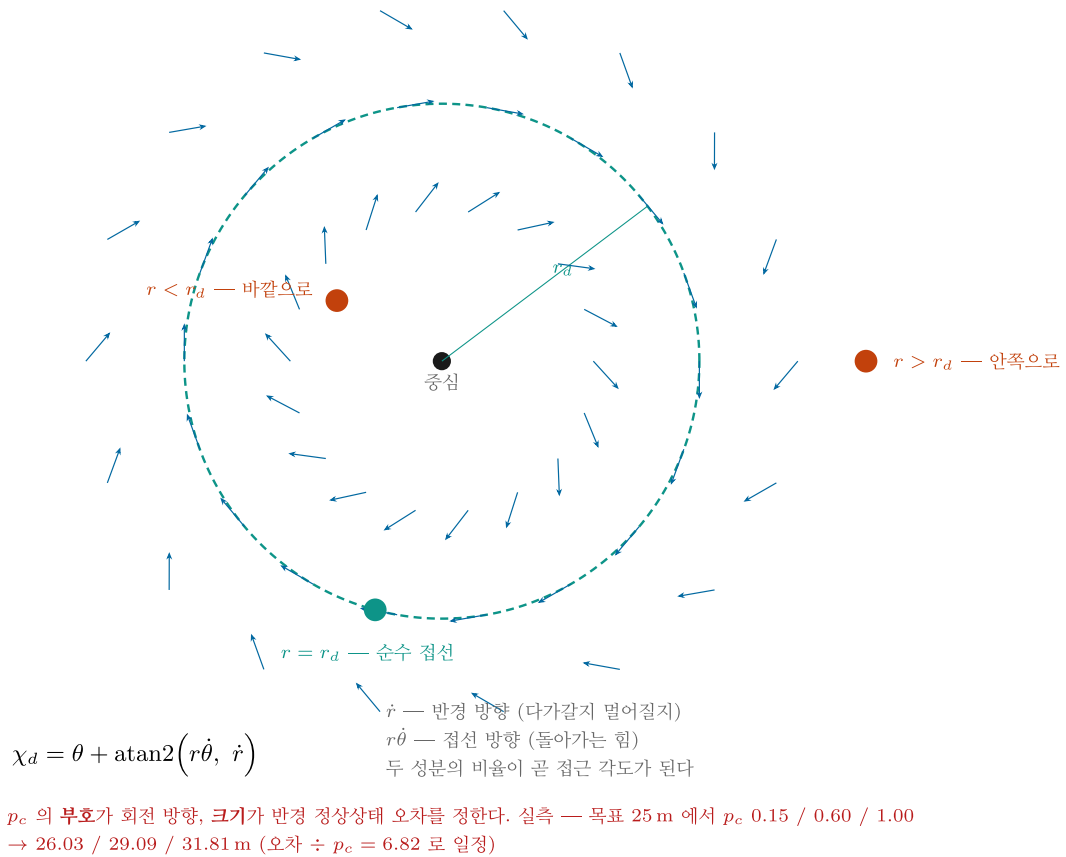
- 원 위에 점을 촘촘히 찍고 7주차 LOS 로 따라가면 되지 않을까?

⚠ 세 가지 조건이 관련된다

| 문제       | 내용                                                |
|----------|---------------------------------------------------|
| 점 개수     | 반경 25 m 원을 1 m 오차로 근사하려면 약 25개. 반경이 바뀌면 다시 만들어야 함 |
| 모서리      | 다각형이므로 꼭짓점마다 방향이 꺾임. 7주차에서 본 코너 이탈이 매 점마다 생김      |
| 방향·반경 변경 | 실시간으로 바꾸려면 웨이포인트 배열을 통째로 다시 만들어야 함                |

- 원은 직선의 모음이 아니다. 원을 원으로 다루는 유도법칙이 필요하다

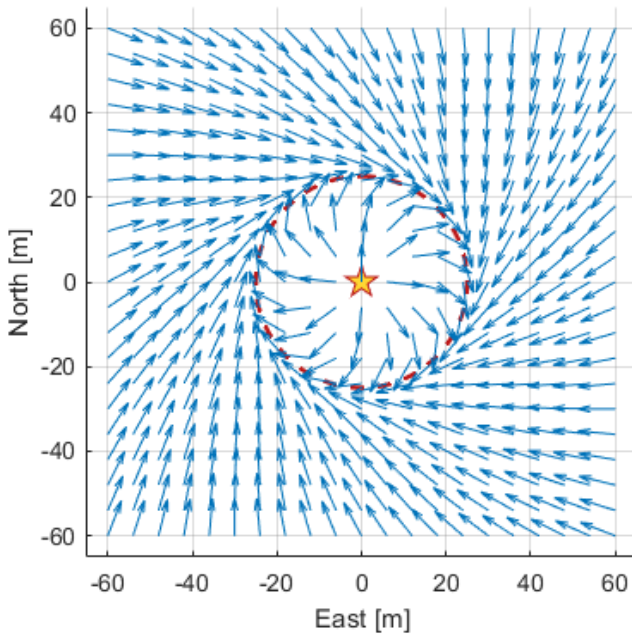
### 1-3. 벡터필드라는 발상



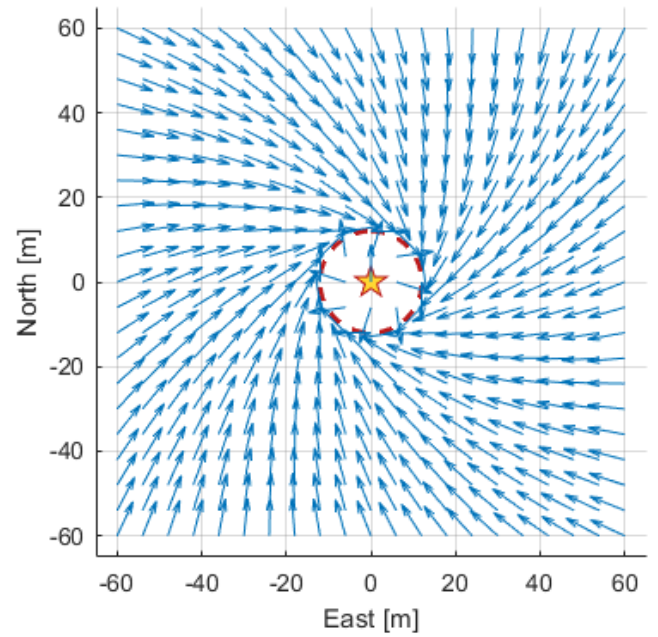
#### ★ "경로를 따라간다" 대신 "공간에 방향을 깔아 둔다"

- LOS 는 배의 위치를 보고 그때그때 목표 방향을 계산했다
- 벡터필드는 반대로 생각한다
  - 공간의 모든 점에 "여기서는 이쪽으로 가라"는 화살표를 미리 깔아 둬
  - 배는 자기가 있는 자리의 화살표를 그대로 따라감

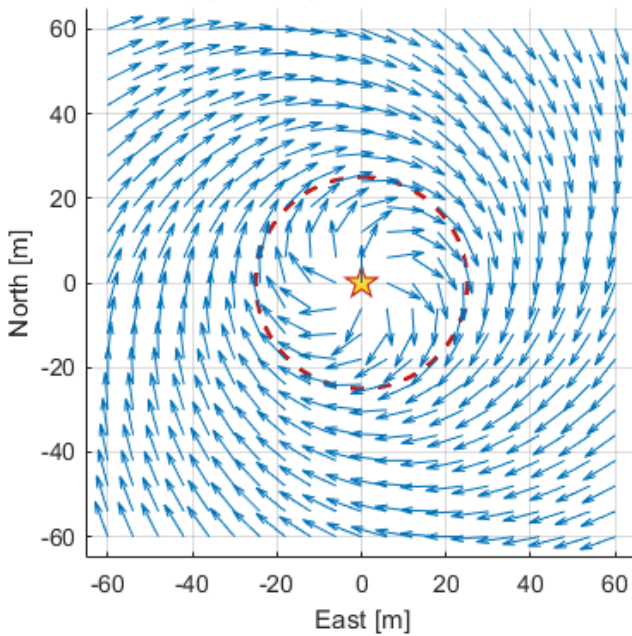
1) 기본  $r_d = 25$ ,  $p_c = +0.313$  (시계방향)



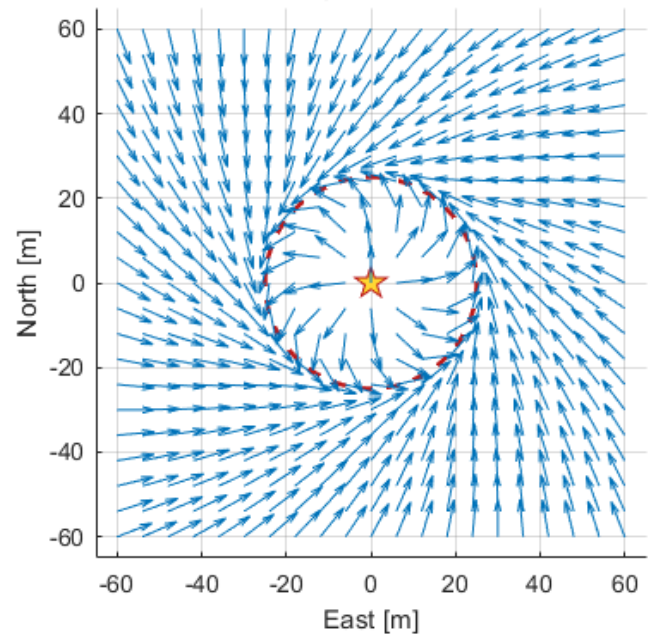
2) 반경 축소  $r_d = 12$



3)  $|p_c|$  증가  $p_c = 1.5$  (원에 덜 끌린다)



4) 부호 반전  $p_c = -0.313$  (반시계방향)



## 필드 읽는 법

| 위치   | 화살표가 향하는 곳              |
|------|-------------------------|
| 원 바깥 | 중심 쪽으로 기울어 있음 → 원으로 다가감 |
| 원 위  | 원에 접함 → 그대로 둠           |
| 원 안쪽 | 바깥으로 기울어 있음 → 원으로 밀려남   |

- 어디서 출발하든 결국 원 위로 모인다
- 이 성질을 수렴성(convergence) 이라고 한다

### ! 벡터필드는 원 말고도 쓴다

직선 경로에도 쓰고, 장애물 회피에도 쓴다 (11주차 인공포텐셜장이 같은 계열이다).  
이번 주차의 학습 대상은 그중 원 궤도용 하나다.

## 1-4. 논문의 식 (9) — 극좌표 벡터필드

출처: W. Jung, S. Lim, D. Lee, H. Bang,

"Unmanned Aircraft Vector Field Path Following with Arrival Angle Control"

— 50-원본자료 / 에 PDF 있음. KAIST 항공우주공학과

### 먼저 극좌표로 바꾼다

- 중심에서 배까지의 거리  $r$  과 방위  $\theta$  로 위치를 표현

$$\begin{aligned} dN &= N - N_c & dE &= E - E_c \\ r &= \sqrt{(dN)^2 + (dE)^2} \\ \theta &= \text{atan2}(dE, dN) & \text{NED 이므로 } \text{atan2}(\text{East}, \text{North}) \end{aligned}$$

### 식 (9)

$$\dot{r} = \frac{-v_d(r - r_d)}{\sqrt{(r - r_d)^2 + (p_c r)^2}}, \quad r\dot{\theta} = \frac{v_d p_c r}{\sqrt{(r - r_d)^2 + (p_c r)^2}}$$

| 기호              | 뜻                     |
|-----------------|-----------------------|
| $v_d$           | 원하는 속도 [m/s]          |
| $r_d$           | 원하는 반경 [m]            |
| $p_c$           | 유도 파라미터. 이번 주차의 핵심 대상 |
| $\dot{r}$       | 반경 방향 속도 — 원에 붙는 성분   |
| $r\dot{\theta}$ | 접선 방향 속도 — 도는 성분      |

### 두 성분이 하는 일

| 조건              | $\dot{r}$ | $r\dot{\theta}$ | 결과           |
|-----------------|-----------|-----------------|--------------|
| $r > r_d$ (바깥)  | 음수        | 있음              | 중심 쪽으로 좁히며 돌 |
| $r = r_d$ (원 위) | 0         | 최대              | 순수하게 돌       |
| $r < r_d$ (안쪽)  | 양수        | 있음              | 바깥으로 넓히며 돌   |

### 분모가 하는 일

분모는 두 성분을 벡터로 합쳤을 때의 크기다.

그래서 합성 속도의 크기가 항상  $v_d$  가 된다. 어디서든 같은 속도로 움직인다.

## 1-5. 식 (13) — 어느 방향으로 갈 것인가

$$\chi_d = \theta + \text{atan2}(r\dot{\theta}, \dot{r})$$

- $\theta$  는 중심에서 배를 본 방위, 거기에 필드가 시키는 상대 각도를 더함
- 본 과목의 구현은 극좌표를 NED 로 먼저 풀고 각도를 뽑는다. 같은 식이지만 특이점이 없다

```
vfN = r_dot*cos(theta) - rth_dot*sin(theta);
vfE = r_dot*sin(theta) + rth_dot*cos(theta);
chi_d = atan2(vfE, vfN);
```

### ! 연구실 UGV 구현과 같은 형태다

UGV\_Control\_StateFlow\_260529.slx 의 KAIST\_CircularVF 함수와 같다.  
W08\_matlab/Guid\_VF\_Loiter.m 은 이것을 한 줄로 압축한 것이다.

```
psi_ref = theta + atan2(vf_pc*r, -(r - vf_rd));
```

두 식이 같음을 손으로 확인해 볼 것 (과제 ①).

## 1-6. p\_c — 부호와 크기

### ★ 부호가 회전 방향을, 크기가 원에 끌리는 정도를 정한다

#### 부호 = 방향

논문 III절 원문

"Positive and negative parameters generate **clockwise** and **counterclockwise** vectors around the origin, respectively."

| p_c | 회전 방향 |
|-----|-------|
| > 0 | 시계방향  |
| < 0 | 반시계방향 |

- 확인: 원 위 정북 지점( $\theta = 0$ )에서  $p_c > 0$  이면 속도가 **동쪽**을 향함  
→ 위에서 오른쪽으로 = 시계방향

### ✕ 참고 코드의 한글 주석이 반대로 적혀 있다

W08\_matlab/VF\_test\_code\_main.m 에 "양수: 반시계 방향 회전" 이라고 되어 있으나  
논문 본문과 수치 확인 결과 **양수가 시계방향**이다. 코드 주석보다 원문과 실험을 믿을 것.

#### 크기 = 수렴 성향

논문 원문

"A particle on the field with the **smaller** |p\_c| is pulled **more** toward the loitering circle."

| |p\_c| | 필드 모양 | 결과 |

---|---|---

| **작다** | 화살표가 원 쪽으로 강하게 꺾임 | 빨리 붙지만 급함 |

| **크다** | 화살표가 소용돌이처럼 완만 | 천천히 붙음 |

- 위 그림의 3번 패널이  $|p_c| = 1.5$  인 경우다. 원 밖에서도 거의 돌기만 한다

## 1-7. $p_c$ 를 계산해서 정하기 — 식 (27)·(28)

★ 눈대중으로 고르지 않는다. 논문이 설계식을 준다

- 헤딩 추종에 시간지연  $\tau_r$  이 있을 때 반경 오차의 특성방정식 — 식 (27)

$$s^2 + \frac{1}{\tau_r} s + \frac{v}{p_c r_d \tau_r} = 0$$

- 표준형  $s^2 + 2\zeta\omega_n s + \omega_n^2$  과 비교하면 — 식 (28)

$$\omega_c = \sqrt{\frac{v}{p_c r_d \tau_r}}, \quad \zeta_c = \frac{1}{2} \sqrt{\frac{p_c r_d}{v \tau_r}}$$

- 원하는 감쇠비  $\zeta_c$  로 뒤집으면

$$p_c = \frac{4\zeta_c^2 v \tau_r}{r_d}$$

### 대상 선박에 적용

| 항목        | 값       | 어디서                                                          |
|-----------|---------|--------------------------------------------------------------|
| $v$       | 1.5 m/s | 목표 속도                                                        |
| $r_d$     | 25 m    | 목표 반경                                                        |
| $\tau_r$  | 1.31 s  | 헤딩 루프 $1/(\zeta\omega_n)$ , $\omega_n=0.783$ , $\zeta=0.978$ |
| $\zeta_c$ | 1       | 오버슈트 없이 붙게                                                   |

$$p_c = \frac{4 \times 1^2 \times 1.5 \times 1.31}{25} = 0.313$$

! 논문의 0.33 을 그대로 쓰면 안 된다

논문 값은  $v = 25$  m/s,  $r_d = 150$  m,  $\tau_r = 0.5$  s 인 무인기 기준이다.  
우연히 숫자가 비슷하지만 근거가 다르다. 자기 배의 값으로 다시 계산할 것.

## 1-8. 반경 오차는 왜 남는가

★ 원 위에 정확히 올라서지 않는다. 항상 바깥쪽으로 조금 밀린다

- 기준 환경 실측값 ( $r_d = 25$  m,  $v = 1.5$  m/s)

| $p_c$        | 정착 반경 [m]    | 오차 [m]       |
|--------------|--------------|--------------|
| 0.150        | 26.03        | +1.03        |
| <b>0.313</b> | <b>27.14</b> | <b>+2.14</b> |
| 0.600        | 29.09        | +4.09        |
| 1.000        | 31.81        | +6.81        |

- 오차가  $p_c$  에 정확히 비례한다 (오차 ÷  $p_c$  = 6.82 로 일정)



## 왜 그런가 — 논문 식 (29)~(31)

- 원을 돌면 지령 각도  $\chi_d$  가 계속 회전한다. 회전율은  $v/r_d$
- heading 제어기는 이렇게 계속 움직이는 지령을 항상 조금 늦게 따라간다
- 늦은 만큼 배가 바깥으로 밀린다

$$r_{ss} - r_d \approx p_c v \tau_{\text{eff}}$$

- 실측에서  $\tau_{\text{eff}} = 6.82 / 1.5 = 4.55 \text{ s}$

## 지연의 정체 세 가지

| 기여                   | 등가 지연  | 근거                                            |
|----------------------|--------|-----------------------------------------------|
| heading 루프의 램프 추종 오차 | 2.50 s | $(K_d + N r)/K_p = 1000/400$ . 적분기가 없어 생기는 오차 |
| 모터 지연                | 0.30 s | $\tau_n$                                      |
| 크랩각                  | 1.93 s | 정상선회에서 $\beta \approx -5.8^\circ$             |
| 합                    | 4.73 s | 실측 4.55 s                                     |

### ★ 크랩각이 또 나온다

벡터필드가 주는 것은 속도가 향할 방향(침로  $\chi$ ) 이다.  
 그런데 배에는 뱃머리 방향(선수각  $\psi$ ) 밖에 지시할 수 없다.  
 선회 중에는 둘이  $\beta$  만큼 어긋나고, 그것이 그대로 반경 오차가 된다.  
 → 7주차 [조류가-있으면-뱃머리와-진행방향이-다르다](#) 와 같은 문제다

## 줄이는 법

| 방법                                          | 효과 (실측)                             |
|---------------------------------------------|-------------------------------------|
| $p_c$ 를 줄인다                                 | 오차가 비례해서 줄지만 수렴이 느려짐                |
| 크랩각 보상 $\psi_{\text{ref}} = \chi_d - \beta$ | 2.14 → 1.24 m (42 % 감소)             |
| heading 루프를 빠르게                             | $K_p$ 를 올리면 $(K_d + N r)/K_p$ 가 줄어듦 |
| 반경을 키운다                                     | 회전율 $v/r_d$ 가 줄어 지연 영향이 작아짐         |

- 크랩 보상 후 남은 1.24 m 는 이론값  $0.313 \times 1.5 \times 2.8 = 1.31 \text{ m}$  와 거의 같다

## 1-9. 회전수 세기와 임무 종료

- 로이터링은 조치하지 않으면 영원히 돈다. 끝내는 조건이 필요하다

```
d = theta - theta_prev;
d = atan2(sin(d), cos(d));      % +pi -> -pi 로 넘어갈 때를 처리
accum = accum + d;
turns = abs(accum) / (2*pi);
```

### ⚠ 회전수는 원에 도달한 이후부터 계수한다

접근하는 동안에도  $\theta$  는 변한다. 그것까지 세면 한 바퀴를 돌기도 전에 카운트가 올라간다.  
 그래서  $|r - r_d|$  가 허용범위 안에 들어온 순간부터 세기 시작한다.  
 연구실 UGV 예제 `calculate_turns` 도 같은 방식이다.

- `turns ≥ required_turns` 가 되면 속도 지령을 0 으로 만들어 정지
- 이것이 MILS 예제의 `StartMission2 → StopMission2` 전이 조건과 같다

```

PrepareFirst1 → StartMission1 → StartMission2 → StopMission2
(준비)         (웨이포인트)   (로이터링)   (정지)
                        ↑
                number_of_turn >= required_number_of_turn

```

#### ! Stateflow 로 묶는 것은 나중에 한다

이번 주차에는 **로이터링 한 동작만** 만든다.

웨이포인트 항주와 로이터링을 상태기계로 잇는 것은 13주차 미션 매니저에서 다룬다.

## 2부 · 실습

### 실행하면 이런 결과가 나온다

#### ★ 아래는 기준 환경에서 실제로 받은 출력이다

값이 크게 다르면 설정 파일을 먼저 확인한다. 소수점 마지막 자리는 달라질 수 있다.

```
cd('<배포 폴더>/W08_simulink')
W08_setup
out = sim('W08_0_offline');
S = W08_plot(out);
```

#### • 정상 출력

W08 설정 완료

로이터 중심 (N,E) = (20, 40), 반경 25 m

p\_c = +0.313 -> 시계방향, 목표 속도 1.5 m/s

2 바퀴 돌면 정지

===== 시계방향 / r\_d = 25 m / u = 1.5 m/s =====

원 도달 시각 20.6 s

정착 후 평균 반경오차 2.136 m <- 과제에 쓸 값

정착 후 최대 반경오차 2.263 m

반경오차 표준편차 0.044 m

총 회전수 2.00 바퀴

정지 시각 246.8 s

정상상태 속도 1.500 m/s

| 읽는 법            | 뜻                                           |
|-----------------|---------------------------------------------|
| 평균 반경오차 2.136 m | 0 이 되지 않는다. 이유는 §1-8 — p_c 에 비례하는 정상상태 오차다  |
| 표준편차 0.044 m    | 오차가 흔들리지 않고 일정하게 남는다. 잡음이 아니라 편향이라는 뜻       |
| 총 회전수 2.00 바퀴   | 회전수 세기가 정확하다. 1.9 나 2.1 이 나오면 §1-9 를 다시 볼 것 |
| 정상상태 속도 1.500   | 목표와 일치                                      |

#### ! 반경오차가 남는 것이 정상이다

벡터필드는 비례 제어이므로 정상상태 오차가 남는다.

없애려면 적분을 넣어야 하지만, 그러면 진입할 때 넘어간다(overshoot).

§C 에서 p\_c 를 바꿔 이 관계를 직접 확인한다.

### A. 벡터필드 그려 보기 (15분)

- 배를 돌리기 전에 필드부터 눈으로 본다

```
cd('<배포 폴더>/W08_simulink')
W08_vectorfield
```

- 네 조건이 한 번에 나온다

| 패널 | 조건                              | 확인할 것                    |
|----|---------------------------------|--------------------------|
| 1  | <code>r_d=25, p_c=+0.313</code> | 기본. 원 밖은 안쪽으로, 원 안은 바깥으로 |
| 2  | <code>r_d=12</code>             | 원이 작아지고 필드가 따라 바뀜        |
| 3  | <code>p_c=1.5</code>            | 소용돌이가 커짐 — 원에 덜 끌림       |
| 4  | <code>p_c=-0.313</code>         | 화살표가 반대로 돌               |

## 해 볼 것

- **CASES** 배열의 `p_c` 를 0.05 로 바꿔 볼 것. 원 밖에서 거의 직선으로 중심을 향한다
- 원 위 정북 지점(`E=0, N=r_d`)의 화살표가 **동쪽**을 향하는지 확인 → 시계방향

## B. 오프라인 로이터링 (30분)

W08\_setup

★ 7주차와 마찬가지로 설정 파일을 실행하면 곧바로 돈다

- 정상 출력

W08 설정 완료

로이터 중심 (N,E) = (20, 40), 반경 25 m  
`p_c = +0.313` -> 시계방향, 목표 속도 1.5 m/s  
 2 바퀴 돌면 정지

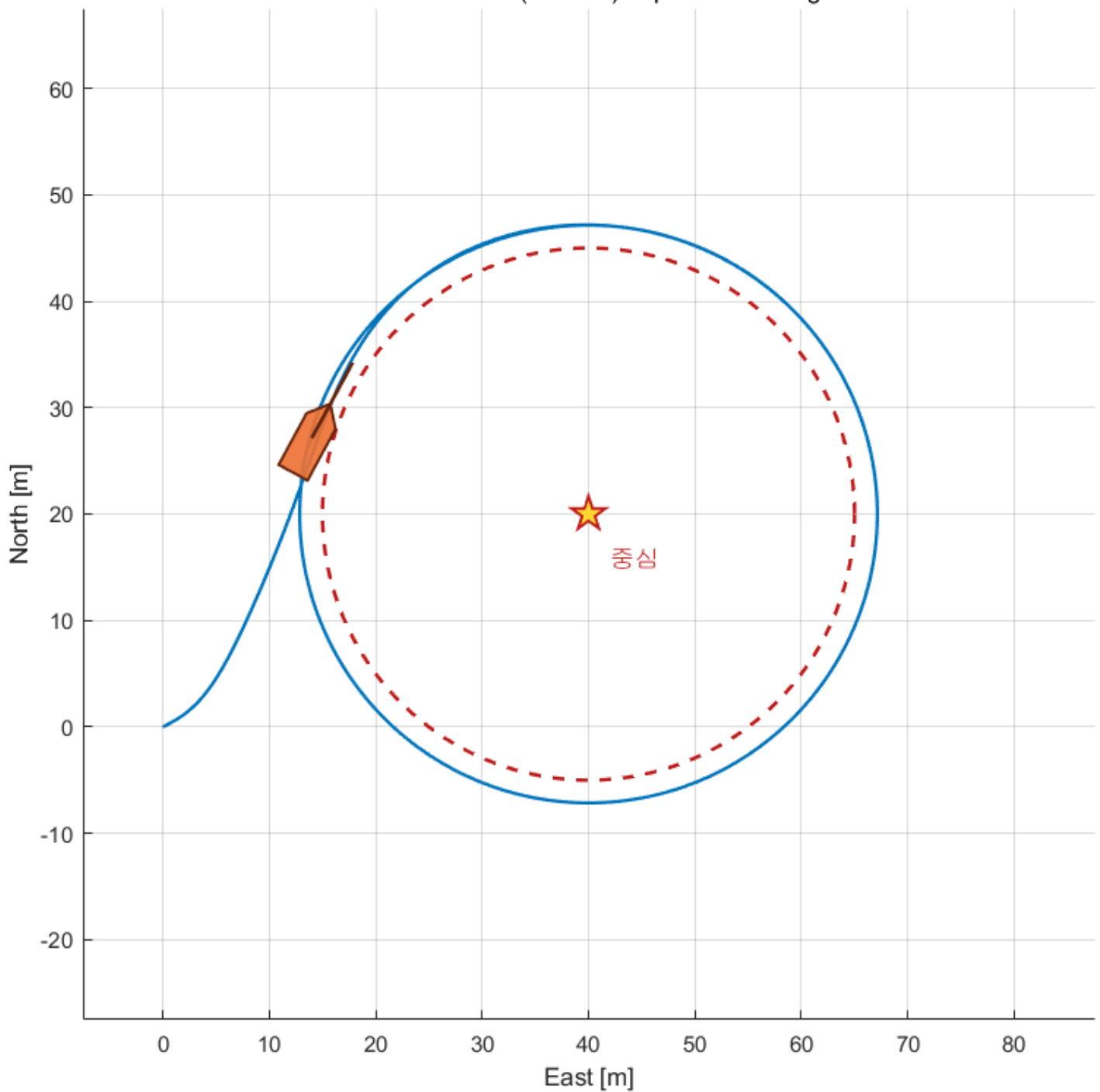
===== 시계방향 / `r_d = 25 m` / `u = 1.5 m/s` =====

|              |           |            |
|--------------|-----------|------------|
| 원 도달 시각      | 20.6 s    |            |
| 정착 후 평균 반경오차 | 2.136 m   | <- 과제에 쓸 값 |
| 정착 후 최대 반경오차 | 2.263 m   |            |
| 반경오차 표준편차    | 0.044 m   |            |
| 총 회전수        | 2.00 바퀴   |            |
| 정지 시각        | 246.8 s   |            |
| 정상상태 속도      | 1.500 m/s |            |

## 실시간 창

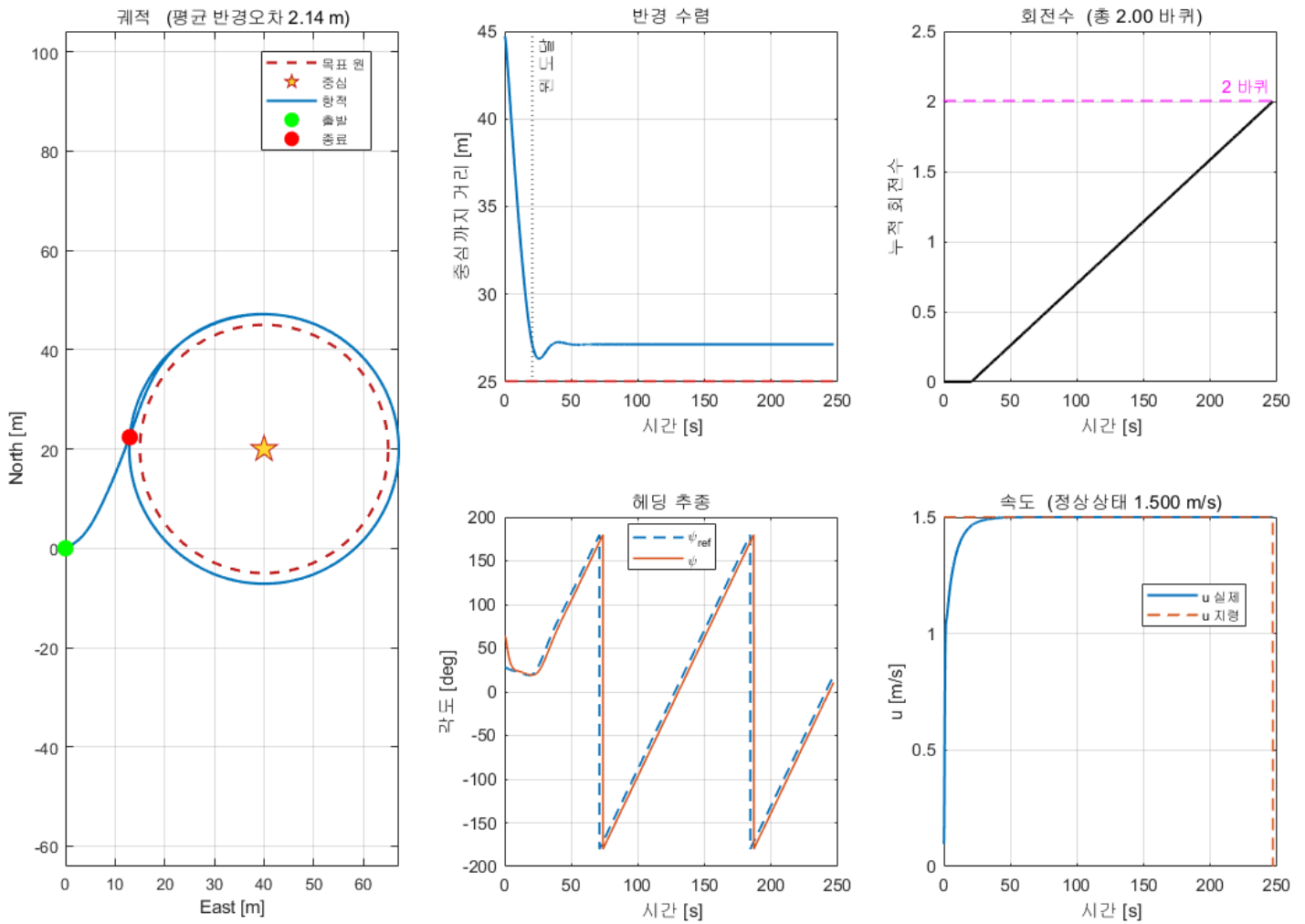
W08 로이터링 (선수 = 세모, 선미 = 네모)

t = 800.0 s   r = 26.96 m (목표 25)   psi = 28.4 deg   2.03 바퀴



- 빨간 점선이 목표 원, 별표가 중심
- 제목 줄에 **현재 반경**과 **누적 회전수**가 표시됨
- 배가 바깥에서 접근해 원에 붙고, 2바퀴 돈 뒤 멈추는 것을 확인할 것

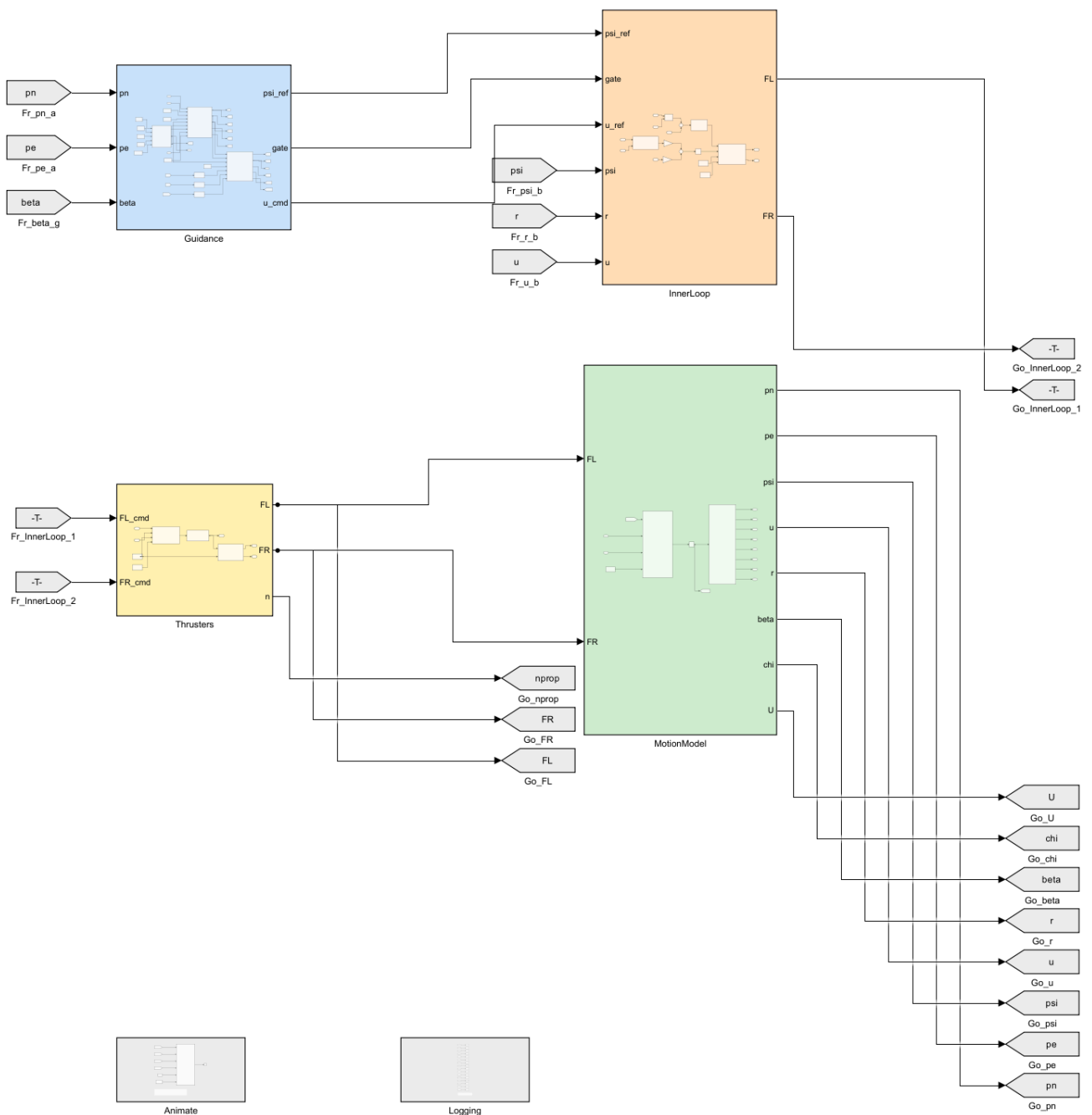
## 성능 그림



| 그림    | 볼 것                                     |
|-------|-----------------------------------------|
| 궤적    | 목표 원(점선)과 실제 항적(실선)의 간격                 |
| 반경 수렴 | 오버슈트 없이 붙는가. <b>일정한 오차가 남는가</b>         |
| 회전수   | 원 도달 전에는 0, 도달 후 선형 증가                  |
| 헤딩 추종 | $\psi_{ref}$ 가 계속 회전하고 $\psi$ 가 조금 뒤따라감 |
| 속도    | 1.5 m/s 유지, 정지 시 0                      |

## 모델 구조

W08 실습 A — 오프라인 WAM-V 로이더링 (Gazebo 필요)  
 LoiterVF -> TurnCount -> Inner Loop -> Thrusters -> 운동모델  
 유도만 7주차와 다르다. 나머지는 그대로다.  
 실행 전 >> W08\_setup



| 단 | 블록                                            | 7주차 대비           |
|---|-----------------------------------------------|------------------|
| 1 | <code>Schedule</code> → <code>LoiterVF</code> | 바뀜 — LOS 대신 벡터필드 |
| 1 | <code>TurnCount</code>                        | 추가 — 회전수와 종료 판정  |
| 2 | <code>InnerLoop</code>                        | 그대로              |
| 3 | <code>Thrusters</code>                        | 그대로              |
| 4 | <code>MotionModel</code>                      | 그대로              |
| — | <code>Animate</code>                          | 원과 회전수만 추가       |

- `LoiterVF` · `TurnCount` · `Schedule` 은 `Guidance` 상자 안에 있다. 더블클릭해서 열 것
- 로이터 중심·반경·회전수 같은 설정값도 그 안에 있다. 최상위에는 신호만 보인다
- `Logging` 과 `Animate` 는 입출력 포트가 없는 상자. 안에서 From 태그로 받는다

## 블록 색으로 역할을 구분한다

! 7~9주차의 모든 모델이 같은 색 규칙을 따른다

| 색   | 역할            | 예                                                                          |
|-----|---------------|----------------------------------------------------------------------------|
| 파랑  | 유도 (Guidance) | <code>Guidance</code> , <code>WpManager</code> , <code>LoiterVF</code>     |
| 보라  | 미션 판단         | <code>MissionFSM</code> , <code>ModeSwitch</code> , <code>TurnCount</code> |
| 주황  | 제어 (Control)  | <code>InnerLoop</code> , PID, 추력배분                                         |
| 노랑  | 추진기           | <code>Thrusters</code>                                                     |
| 초록  | 운동모델 (Plant)  | <code>MotionModel</code>                                                   |
| 연보라 | ROS 통신        | <code>PoseSubscriber</code> , <code>CmdPublisher</code>                    |
| 회색  | 로깅·표시         | <code>Logging</code> , <code>Animate</code> , Goto/From 태그                 |
| 흰색  | 설정값           | Constant                                                                   |

- 선은 직선 또는 직각만 씀. 대각선 없음
- 되먹임처럼 뒤로 돌아가는 선만 꺾임
- 배치를 흐트러뜨렸으면 생성 스크립트를 다시 돌린다 (배치까지 다시 만든다)

```
w08_setup
build_w08_models
```

## C. 방향·속도·반경 바꾸기 (30분)

### C-1. 하나씩 바꿔 보기

- `w08_setup.m` 에서 값을 고치고 다시 실행



| 바꿀 값           | 시도               | 예상                |
|----------------|------------------|-------------------|
| p_c            | -0.313           | 반시계방향으로 둠         |
| p_c            | 0.15 / 0.6 / 1.0 | 반경 오차가 비례해서 커짐    |
| r_d            | 12 / 40          | 원 크기가 바뀜          |
| u_ref          | 1.0 / 2.0        | 도는 속도가 바뀜. 오차도 비례 |
| required_turns | 0                | 멈추지 않고 계속         |
| crab_comp      | 1                | 반경 오차가 절반 가까이 줄어듦 |

## C-2. 도는 도중에 바꾸기

- use\_schedule = 1 로 두면 시간에 따라 자동으로 바뀐다

| 구간        | 방향   | 속도          | 반경        |
|-----------|------|-------------|-----------|
| 0~150 s   | 설정대로 | u_ref       | r_d       |
| 150~300 s | 반전   | u_ref       | r_d       |
| 300~450 s | 반전   | u_ref × 1.6 | r_d       |
| 450 s~    | 반전   | u_ref × 1.6 | r_d × 0.6 |

- 기준 환경 실측값 ( crab\_comp = 1 , required\_turns = 0 )

| 구간         | 평균 반경 [m] | 평균 u [m/s] | 회전 방향 |
|------------|-----------|------------|-------|
| 1) 기본      | 26.24     | 1.500      | 시계방향  |
| 2) 방향 반전   | 26.24     | 1.500      | 반시계방향 |
| 3) 속도 1.6배 | 28.47     | 2.180      | 반시계방향 |
| 4) 반경 0.6배 | 18.77     | 2.127      | 반시계방향 |

### ⚠ 3번 구간에서 속도가 목표(2.4)에 못 미친다

2.4 m/s 를 유지하려면  $X = (100+150 \times 2.4) \times 2.4 = 1104 \text{ N}$  , 편당 552 N 이 필요한데  $F_{\max} = 500$  이라 포화한다. 게다가 선회에도 추력을 나눠 써야 한다.  
7주차에서 본 것과 같은 문제다 — 게인이 아니라 포화가 한계다.

### ⚠ 4번 구간에서 오차가 크게 는다 (+3.77 m)

반경이 작아지면 회전을  $v/r_d$  가 커지고, 그만큼 헤딩 지연의 영향이 커진다.  
작은 원을 빠르게 도는 것이 가장 어렵다.

## D. VRX 연동 (35분)

- 7주차 E절과 절차가 완전히 같다. 스크립트 이름만 바뀐다

```
ros2 launch vrx_gz competition.launch.py \
  world:=sydney_regatta \
  urdf:=$HOME/capstone_ws/wamv/w7_wamv.urdf.xacro
```

★ 7주차 주행이 끝난 상태에서 그대로 이어 돌리지 않는다

배가 경로 끝(원에서 100 m 밖)에 있으면 **원 도달에 실패**한다.

실측에서 이 상태로 돌렸더니 회전수 0, 반경오차 37.9 m 가 나왔다.

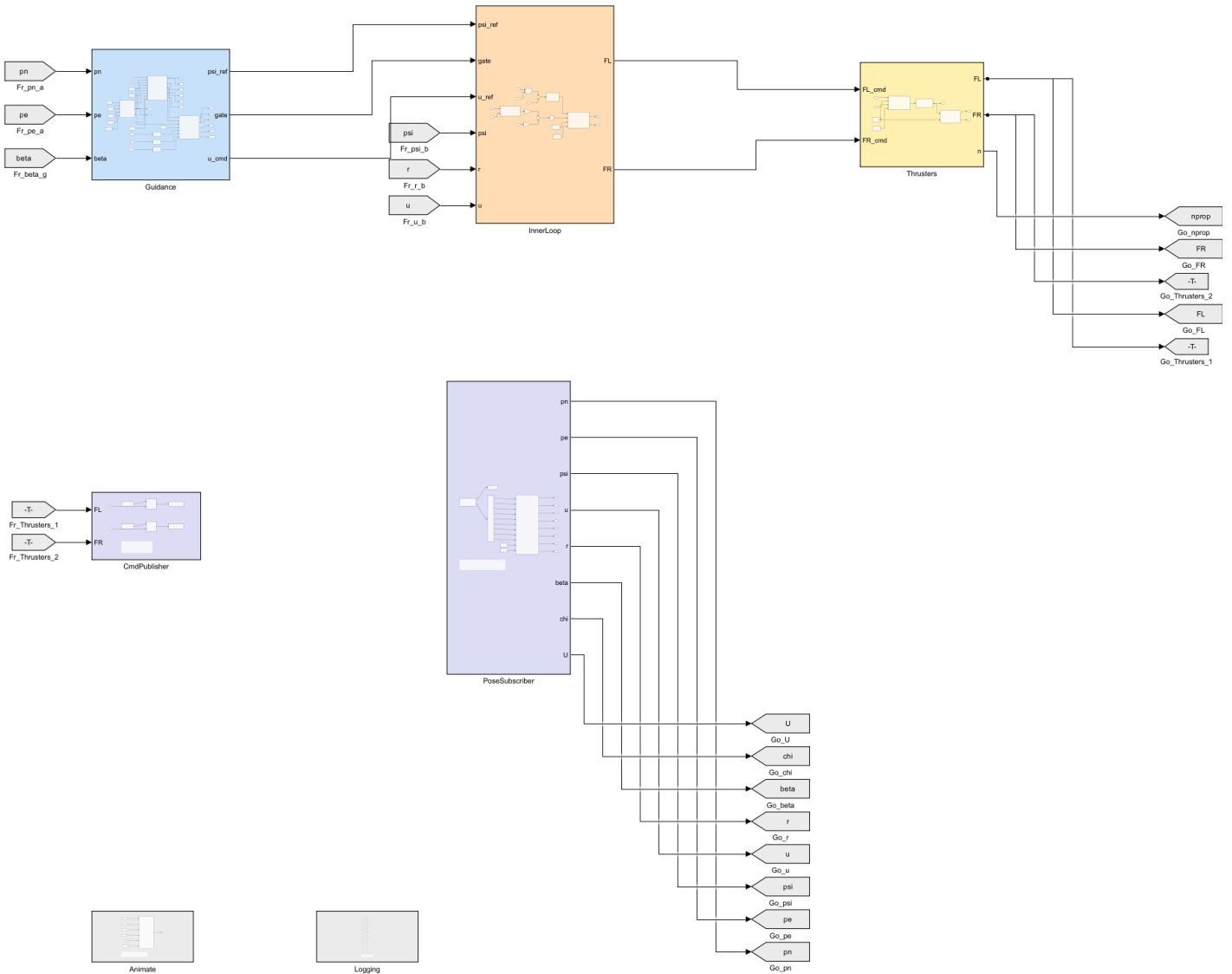
**VRX** 를 꺾다가 다시 띄워 배를 스폰 위치로 되돌린 뒤 시작한다.

RTF 가 1 % 미만인 노트북은 `"extra_gz_args:---render-engine-server ogre"` 를 함께 준다.

w08\_setup

w08\_vrx\_run

- `w08_vrx_run.m` 이 토픽 확인 → RTF 측정 → 페이싱 설정 → 실행 → 그림까지 한다
- 도는 동안 **실시간 그림**이 뜬다 (VRX 모델에도 `Animate` 블록이 있다)



## ✗ 로이더 원이 수역 안에 들어가야 한다

스폰 기준 안전 수역은  $N \in [-28, 60]$ ,  $E \in [-18, 100]$  이다.

기본 설정(중심 (20,40), 반경 25)이면 원이  $N \in [-5, 45]$ ,  $E \in [15, 65]$  를 차지해 안전하다.

반경을 40 이상으로 키우면 **육지에 얹힌다**.

## D-1. 결과

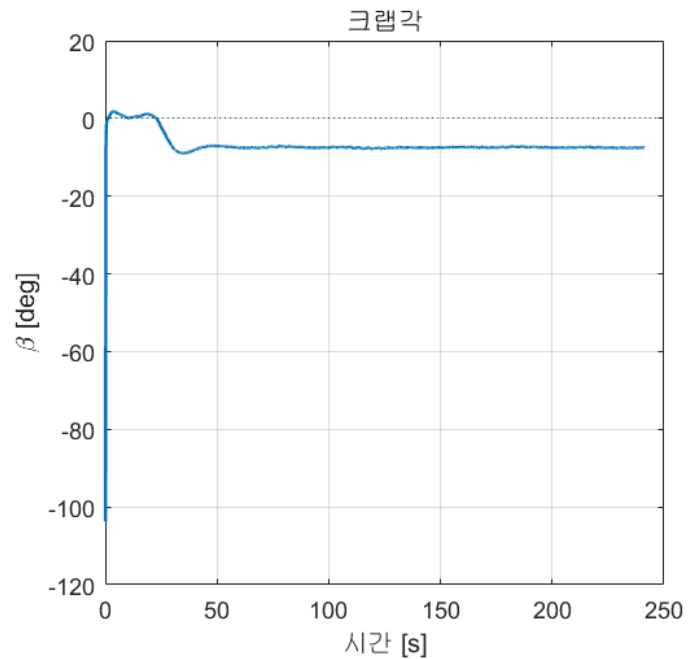
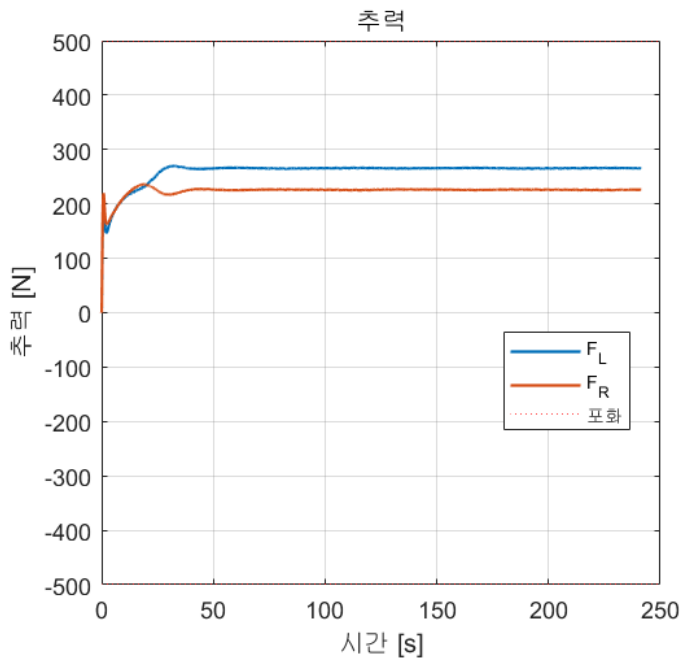
- 기준 환경 실측값 (2026-09-04, RTF 0.472, 300초)

|              |           |
|--------------|-----------|
| 원 도달 시각      | 21.6 s    |
| 정착 후 평균 반경오차 | 2.268 m   |
| 정착 후 최대 반경오차 | 2.442 m   |
| 반경오차 표준편차    | 0.055 m   |
| 총 회전수        | 2.00 바퀴   |
| 정지 시각        | 247.9 s   |
| 정상상태 속도      | 1.500 m/s |

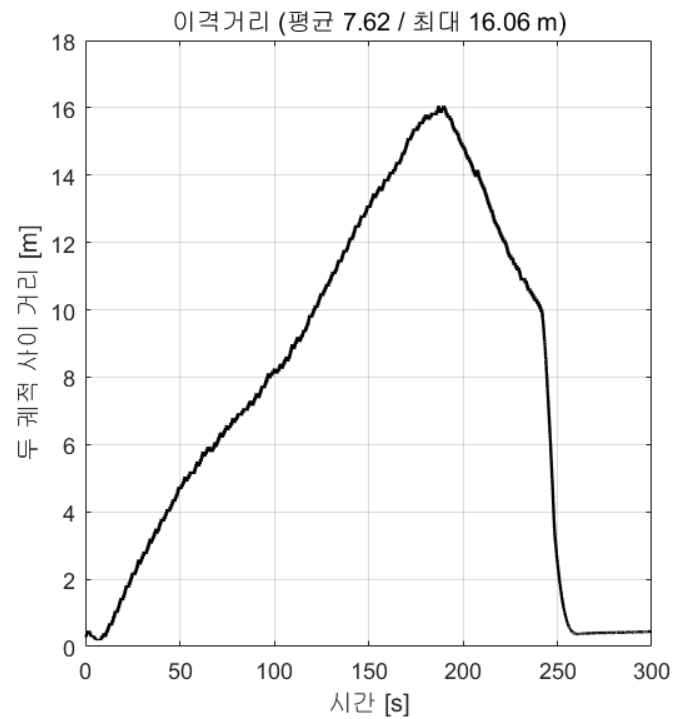
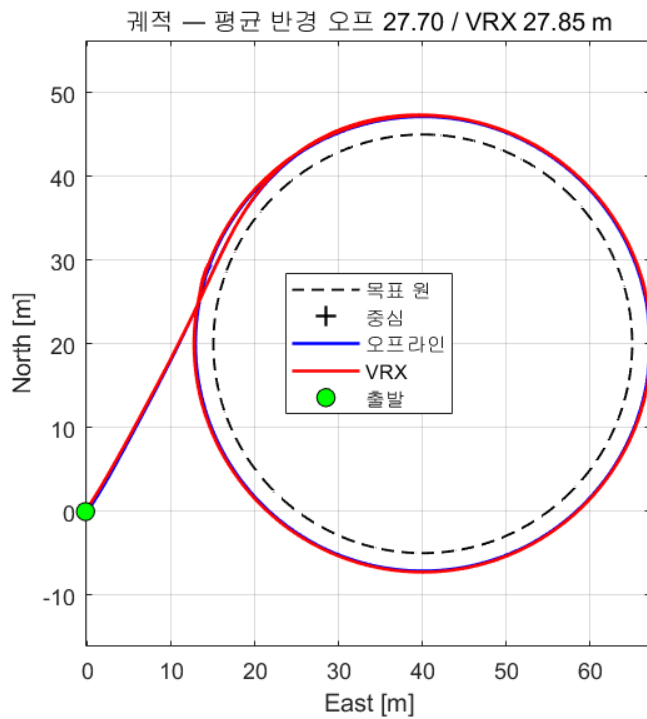
- 다른 노트북 재측정 (2026-09-15, RTF 0.843, 300초)

|              |           |
|--------------|-----------|
| 원 도달 시각      | 20.8 s    |
| 정착 후 평균 반경오차 | 2.269 m   |
| 정착 후 최대 반경오차 | 2.439 m   |
| 반경오차 표준편차    | 0.047 m   |
| 총 회전수        | 2.00 바퀴   |
| 정지 시각        | 241.5 s   |
| 정상상태 속도      | 1.500 m/s |

- 반경오차는 소수점 셋째 자리까지 같다 (2.268 / 2.269). 컴퓨터 성능과 무관한 값이라는 뜻



## D-2. 오프라인과 비교



| 항목               | 오프라인  | VRX   | 차이          |
|------------------|-------|-------|-------------|
| 정착 후 평균 반경오차 [m] | 2.137 | 2.268 | +0.131      |
| 평균 반경 [m]        | 27.70 | 27.85 | +0.15       |
| 회전수              | 2.00  | 2.00  | 0           |
| 정지 시각 [s]        | 250.6 | 247.9 | -2.7        |
| 궤적 평균 이격 [m]     |       |       | <b>0.31</b> |
| 궤적 최대 이격 [m]     |       |       | <b>0.67</b> |

- 다른 노트북 재측정 (2026-09-15, 페이싱 0.843)

| 항목               | 오프라인  | VRX   | 차이           |
|------------------|-------|-------|--------------|
| 정착 후 평균 반경오차 [m] | 2.136 | 2.269 | +0.133       |
| 평균 반경 [m]        | 27.70 | 27.85 | +0.15        |
| 회전수              | 2.00  | 2.00  | 0            |
| 정지 시각 [s]        | 246.8 | 241.5 | -5.3         |
| 궤적 평균 이격 [m]     |       |       | <b>7.62</b>  |
| 궤적 최대 이격 [m]     |       |       | <b>16.06</b> |

### ★ 원 위에서는 이격이 거의 0 이다

7주차의 사각형 경로보다도 잘 겹친다. 이유는 단순하다.

로터링은 정상상태 선회라서 과도응답이 거의 없고,  
두 모델의 차이가 드러나는 순간(급선회·정지·출발)이 없기 때문이다.

### ⚠ 이격 수치는 페이싱 비율이 실측 RTF 와 얼마나 맞았는가에 좌우된다

- 위 두 실측의 반경오차는 2.268 / 2.269 로 같지만 이격은 0.31 m 와 7.62 m 로 다르다
- 원 위를 도는 속도는 같은데 **같은 시각에 원의 다른 지점에** 있기 때문이다
- 판정은 **반경오차 · 회전수 · 정지 시각**으로 한다. 이격은 보조 지표다
- 페이싱 비율을 실측 RTF 보다 **높게** 잡으면 더 나빠진다.  
비율 0.97 로 올려 본 결과, 300초 시뮬레이션이 381초 걸려 실효 비율이 **0.787** 로 떨어지고 회전수가 1.66 바퀴에 그쳤다 (2026-09-15 실측). **스크립트가 켜 값을 그대로 쓴다.**

### 확인할 것

- ☐ 오프라인과 같은 반경 오차가 나오는가
- ☐ 회전수가 같은 속도로 올라가는가
- ☐ **u** 평균 × 시간 ≈ 주행거리 인가 (페이싱이 맞는지)
- ☐ **p\_c** 부호를 뒤집으면 VRX 에서도 반대로 도는가

# 마무리

## 이번 주차 요약

| 순서 | 한 일        | 확인                      |
|----|------------|-------------------------|
| 1  | 벡터필드 그려 보기 | 네 조건의 화살표 모양            |
| 2  | 오프라인 로이터링  | 2바퀴 돌고 정지               |
| 3  | 방향 반전      | <code>p_c</code> 부호 하나로 |
| 4  | 속도·반경 변경   | 시나리오 4단계                |
| 5  | 크랩각 보상     | 오차 2.14 → 1.24 m        |
| 6  | VRX 연동     | 오프라인과 대조                |

## 수업 진도 체크

★ Term Project 를 시작하기 위한 최소 조건

### 이론 이해

- ☐ 로이터링이 무엇이고 왜 필요한지 말할 수 있다
- ☐ 웨이포인트로 원을 근사하면 안 되는 이유 세 가지를 안다
- ☐ 벡터필드가 LOS 와 무엇이 다른지 설명할 수 있다
- ☐ 식 (9) 의  $\dot{r}$  과  $r\dot{\theta}$  가 각각 무슨 역할인지 안다
- ☐ 분모가 있어서 속도 크기가 항상  $v_d$  라는 것을 안다
- ☐ `p_c` 의 부호가 회전 방향임을 안다 (양수 = 시계방향)
- ☐ `p_c` 의 크기가 커지면 원에 덜 끌린다는 것을 안다
- ☐ 식 (28) 로  $p_c = 4\zeta^2 v \tau_r / r_d$  를 계산할 수 있다
- ☐ 반경 오차가 왜 바깥쪽으로 남는지 설명할 수 있다
- ☐ 그 지연의 세 가지 원인을 말할 수 있다
- ☐ 회전수를 셀 때 원 도달 후부터 세야 하는 이유를 안다

### 실습 완료

- ☐ `w08_vectorfield` 로 네 조건의 필드를 확인했다
- ☐ `w08_setup` 실행만으로 배가 돌고 2바퀴 후 멈췄다
- ☐ `p_c` 부호를 바꿔 반시계방향으로 돌렸다
- ☐ `p_c` 크기를 바꿔 반경 오차가 비례하는 것을 확인했다
- ☐ `r_d` 와 `u_ref` 를 바꿔 봤다
- ☐ `crab_comp = 1` 로 오차가 줄어드는 것을 확인했다
- ☐ `use_schedule = 1` 로 도중에 방향·속도·반경이 바뀌는 것을 봤다
- ☐ VRX 에서 로이터링을 실행했다

### 관찰 기록

- ☐ `p_c` 4조건의 정착 반경을 표로 남겼다
- ☐ 크랩 보상 전후를 비교했다

## 과제 8 — 로이터링 성능 분석

- 제출 기한: 9주차 수업 전

### ① 두 구현이 같음을 보이기 (손계산)

- W08\_matlab/Guid\_VF\_Loiter.m 의

```
psi_ref = theta + atan2(vf_pc*r, -(r - vf_rd));
```

- 과 모델 안 LoiterVF 의

```
vfN = r_dot*cos(theta) - rth_dot*sin(theta);
vfE = r_dot*sin(theta) + rth_dot*cos(theta);
chi_d = atan2(vfE, vfN);
```

- 이 둘이 같은 각도를 준다는 것을 유도하시오. (힌트: 분모가 공통이라 약분된다)

### ② p\_c 스윙

- p\_c = 0.15 / 0.313 / 0.6 / 1.0 ( r\_d =25, u\_ref =1.5, crab\_comp =0)

| p_c   | 원 도달 시각 [s] | 정착 반경 [m] | 오차 [m] | 오차/ p_c |
|-------|-------------|-----------|--------|---------|
| 0.15  |             |           |        |         |
| 0.313 |             |           |        |         |
| 0.6   |             |           |        |         |
| 1.0   |             |           |        |         |

- 오차가 p\_c 에 비례하는가? 마지막 열이 일정한지 확인하고 그 값의 의미를 쓰시오
- 식 (28) 로 각 p\_c 의 ζ\_c 를 계산해 표에 함께 넣고, 도달 시각의 경향과 맞는지 논하시오

### ③ 크랩각 보상

- crab\_comp = 0 과 1 을 비교

| crab_comp | 정착 반경 [m] | 오차 [m] | 평균 크랩각 [deg] |
|-----------|-----------|--------|--------------|
| 0         |           |        |              |
| 1         |           |        |              |

- 줄어든 오차가 크랩각으로 설명되는지 수치로 확인하시오  
(힌트: 등가 지연 =  $|\beta| / (v/r)$  )

### ④ 방향·속도·반경 변경

- use\_schedule = 1 로 실행
- 네 구간의 평균 반경·평균 속도·회전 방향을 표로
- 궤적 그림에 구간 경계를 표시

### ⑤ 오프라인 ↔ VRX

- 같은 설정으로 두 모델을 각각 300초 실행



- 궤적을 겹쳐 그리고 반경 오차를 비교

## ⑥ 분석 (8~12줄)

1. `p_c` 를 작게 하면 오차가 줄어든다. 그런데 왜 무한히 작게 하지 않는가?
2. 반경 오차가 항상 **바깥쪽**인 이유를 지연으로 설명하시오
3. 작은 원을 빠르게 도는 것이 어려운 이유는?
4. 로이더링을 조류 속에서 하려면 무엇을 더 해야 하겠는가?

## 평가 기준

| 항목                              | 배점  |
|---------------------------------|-----|
| ① 두 식이 같음을 유도                   | 15% |
| ② <code>p_c</code> 스윙과 비례 관계 확인 | 25% |
| ③ 크랩 보상 효과를 수치로 설명              | 25% |
| ④ 시나리오 실행 및 기록                  | 15% |
| ⑤·⑥ 대조와 분석                      | 20% |

## 막혔을 때

| 증상                                 | 원인                                           | 해결                                                |
|------------------------------------|----------------------------------------------|---------------------------------------------------|
| 먼저 <code>w08_setup</code> 을 실행하십시오 | 워크스페이스에 변수 없음                                | <code>w08_setup</code> 먼저                         |
| 배가 원에 못 붙고 빙빙 돌                    | <code>p_c</code> 가 너무 큼                      | <code>p_c</code> 를 줄일 것                           |
| 원에 붙되 심하게 출렁임                      | <code>p_c</code> 가 너무 작음 ( $\zeta_c < 0.5$ ) | 식 (28) 로 다시 계산                                    |
| 회전 방향이 반대                          | <code>p_c</code> 부호                          | 양수 = 시계방향                                         |
| 회전수가 0 에서 안 올라감                    | 원에 아직 도달 못 함                                 | 반경 오차 그래프 확인                                      |
| 회전수가 너무 빨리 올라감                     | 접근 중에 세고 있음                                  | <code>TurnCount</code> 의 <code>armed</code> 확인    |
| 반경 오차가 5 m 넘게 큼                    | 속도가 빠르거나 반경이 작음                              | <code>u_ref</code> 를 낮추거나 <code>r_d</code> 를 키울 것 |
| 속도가 목표에 못 미침                       | 추력 포화                                        | <code>F_max</code> 확인. 7주차 참조                     |
| VRX 에서 배가 멈춤                       | 육지에 얹힘                                       | 원이 안전 수역 안인지 확인                                   |
| 주행거리와 <code>u</code> 가 2배 차이       | 페이싱 $\neq$ RTF                               | RTF 재측정                                           |

## 참고 자료

### 이 주차의 원본

- W. Jung, S. Lim, D. Lee, H. Bang, "Unmanned Aircraft Vector Field Path Following with Arrival Angle Control" — 50-원본 자료 / PDF. 식 (9), (13), (27), (28)
- 연구실 구현
  - `UGV_Loitering/Guid_VF_Loiter.m` — 압축형 한 줄 구현
  - `UGV_Loitering/Ackermann_Model_YYC/VF_test_code_main*.m` — 필드 시각화 원본

- UGV\_WpFollowingAndLoitering/MILS/UGV\_Control\_StateFlow\_260529.slx  
— KAIST\_CircularVF, calculate\_turns, 4상태 미션 FSM

## 볼트 내 문서

- [W07\\_웨이포인트\\_유도\\_atan2와\\_LOS](#) — 내부루프·추진기·운동모델이 그대로 온다
- [조류가-있으면-벋머리와-진행방향이-다르다](#) — 반경 오차의 크랩각 성분
- [오프라인-모델이-시뮬레이터와-같아야-게인이- 옮겨간다](#) — 왜 오프라인에서 먼저 하는가
- [USV-운동모델은-왜-이렇게-생겼나](#) — 극배치 설계의 배경

## 배포 파일

| 파일                              | 내용                        |
|---------------------------------|---------------------------|
| W08_simulink/W08_setup.m        | 학생이 고치는 설정 파일. 실행하면 바로 돈다 |
| W08_simulink/W08_vectorfield.m  | 벡터필드 네 조건 비교 (실습 A)       |
| W08_simulink/W08_0_offline.slx  | 오프라인 로이터링                 |
| W08_simulink/W08_1_vrx.slx      | VRX 연동                    |
| W08_simulink/W08_plot.m         | 그림 6장 + 성능 지표             |
| W08_simulink/W08_animate.m      | 실시간 그리기                   |
| W08_simulink/build_w08_models.m | 모델 재생성                    |
| W08_matlab/                     | 연구실 원본 (읽기용)              |

## 다음 주 예고

- 9주차 — Stateflow 미션: 웨이포인트와 로이터링을 잇는다
- 7주차의 LOS 와 8주차의 벡터필드를 하나의 임무로 묶는다
- 할 일
  - 유한상태기계(FSM)를 Stateflow 로 그리기
  - WpFollow → Loiter → WpFollow → Finish
  - 두 유도법칙을 **같이 끼우는** 법과 그때 생기는 유의 사항 세 가지
- 준비물
  - 과제 8 의 p\_c 스위치 표
  - 7주차 W07\_setup.m 과 8주차 W08\_setup.m 을 나란히 열어 둘 것 (설정이 합쳐진다)